

01 —

ASSESSMENT SAAS, APP & PLATFORM BUILDING

SolarGrid Energy Solutions Database Design Project — IT212

Stage 1: Requirements Analysis · Stage 2: Conceptual ERD — 15
entities, 20 relationships

// THE VERDICT

// THE SPINE — SINGLE SENTENCE THIS REPORT PROVES

A 15-table, fully normalised MySQL schema with bridge entities for all M:N relationships covers every SolarGrid business operation — design the ERD to 3NF first, then every downstream stage follows without restructuring.

SolarGrid Energy Solutions requires a relational database that tracks customers, sites, equipment inventory, installations, warranties, technicians, service requests, maintenance history, invoices, and payments across a multi-site solar installation business. The binding design constraint is that equipment participates in both the installation lifecycle and the service/warranty lifecycle simultaneously, meaning Equipment sits at the centre of the schema and every major entity either points to it or is derived from work performed on it. Fifteen entities resolve all many-to-many relationships through three bridge tables (InstallationDetail, InstallationAssignment, ServiceAssignment) and satisfy 3NF and BCNF throughout. Stage 1 establishes 20 functional requirements, 15 database-enforced rules, 10 application-enforced rules, and 8 user roles; Stage 2 delivers the full attribute listing, 20 named relationships with cardinality and participation, a written ERD description, and a complete Mermaid erDiagram ready to render.

// KEY FINDINGS

- [1] Eight distinct user roles — Administrator, Sales Rep, Operations Manager, Technician, Warehouse Clerk, Customer Service, Finance Officer, and Auditor — require role-based access control enforced at the application layer, not merely at the database level.
- [2] Equipment is the schema pivot: it carries a status ENUM (Available, Reserved, Installed, Faulty, Decommissioned), links to EquipmentType for catalogue data, to Site when installed, to InstallationDetail for the installation record, to Warranty for coverage tracking, and to ServiceRequest when faults are raised.
- [3] Three M:N relationships — Installation-to-Equipment (resolved by InstallationDetail), Installation-to-Technician (resolved by InstallationAssignment), and ServiceRequest-to-Technician (resolved by ServiceAssignment) — must be modelled as bridge entities with surrogate PKs and composite UNIQUE constraints to prevent duplicate assignments.
- [4] The CompleteInstallation business operation spans three tables (Installation, Equipment, Warranty) and must be implemented as an atomic transaction: partial

completion leaves equipment in an incorrect status and warranties unregistered, both of which corrupt report queries.

[5] Five required reports — warranty expiry, unresolved service requests, technician workload, outstanding customer balances, and revenue by period — drive 12 non-unique indexes on FK and status/date columns and require the schema to avoid derived or computed columns that would force full-table scans.

[6] Invoice.amount_paid is a deliberate controlled denormalisation accepted for query performance; it is maintained transactionally via the Payment table and does not introduce a transitive functional dependency within the Invoice relation itself.

Build depth: 15 entities specified · 20 relationships modelled · 20 functional requirements · 15 DB rules · 10 app rules · Mermaid ERD included

// SECTION INDEX

REF	MODULE	PAGE
01 /	Stage 1 — System roles and 20 functional requirements define the full operational scope	[p.7]
02 /	Stage 1 — NFRs, 15 DB-enforced rules, and 10 app-enforced rules constrain the physical design	[p.11]
03 /	Stage 1 — 15 identified entities with PKs form the schema skeleton	[p.13]
04 /	Stage 2 — Full attribute listings with types, PKs, FKs, and candidate keys per entity	[p.15]
05 /	Stage 2 — 20 named relationships with cardinality, participation, and bridge-entity notes	[p.30]
06 /	Stage 2 — Written ERD description and complete Mermaid erDiagram for direct rendering	[p.32]

What We Are Building and Why the Design Decisions Matter

THE QUESTION & ITS STAKES

SolarGrid Energy Solutions installs and maintains solar power systems for households, schools, farms, and businesses. The company sells physical equipment (panels, batteries, inverters, charge controllers), performs installations across multiple customer sites, assigns technicians to jobs, tracks warranty coverage per individual equipment item, raises service requests when faults occur, and invoices customers for both installation and maintenance work. All of these operations must be captured in a single relational database that enforces data integrity, supports five management reports, and provides an audit trail for every financial transaction.

The design challenge is not complexity in any single entity but in the intersecting lifecycles of one Equipment record: it is received into inventory, reserved for an installation, physically installed at a site, warrantied from its installation date, and potentially the subject of multiple future service requests and maintenance visits — all of which must be queryable independently and joinable together. Getting the Equipment entity and its relationships right in Stage 2 determines whether every downstream stage (normalisation, DDL, stored procedures, reports) is straightforward or requires costly restructuring.

An incorrectly modelled Equipment-to-Installation-to-Warranty chain in Stage 2 propagates into every later stage and cannot be patched without a schema migration.

METHODOLOGY & SOURCE

Requirements were derived from the SolarGrid business scenario: the entities and operations described in the project brief were mapped to functional requirements (FR-01 through FR-20), non-functional requirements, and business rules (DB-01 through DB-15 for database-enforced constraints; APP-01 through APP-10 for application-enforced logic).

Entity identification followed a noun-extraction pass over the requirements, with M:N relationships resolved into bridge entities before the ERD was finalised. All 15 entities carry surrogate INT AUTO_INCREMENT primary keys to simplify FK references; natural candidate keys (email addresses, serial numbers, model numbers, invoice numbers) are preserved as UNIQUE constraints.

Assumptions: Target RDBMS is MySQL 8.0 or MariaDB 10.6+. Team size and concurrent user load are not specified in the brief; the schema is designed for correctness and report performance at small-to-medium scale (hundreds of installations, thousands of equipment items). No external identity provider or multi-tenancy isolation is required for this academic submission.

SCOPE & BOUNDARIES

PERIOD	Academic submission — IT212, September 2026
GEOGRAPHY	South Africa (ZAR currency, South African provinces)
INCLUDED	Stage 1: roles, functional requirements, NFRs, business rules, entity list. Stage 2: entity attributes, relationships, written ERD, Mermaid erDiagram.
EXCLUDED	Stage 3 (normalisation), Stage 4 (DDL), Stage 5 (sample data and reports) are out of scope for this document.
SEGMENTS	Staff (8 roles), Customers (Individual, Business, Government, NGO), Technicians

Bridge / Associative Entity	A table that resolves a many-to-many relationship by holding FKs to both parent tables plus a surrogate PK and any attributes of the relationship itself (e.g. role in team, date assigned).
Candidate Key	Any attribute or minimal set of attributes that uniquely identifies a row and could serve as a primary key; preserved as a UNIQUE constraint when a surrogate PK is used instead.
Participation (Total vs Partial)	Total participation means every instance of the entity must participate in the relationship (double line in Chen notation). Partial participation means some instances may not participate (single line).
Status ENUM	A column constrained to a fixed set of string values enforced at the database level, used throughout the schema to represent lifecycle states (e.g. Equipment status, Installation status, Invoice status).

Produce Stage 1 (Requirements Analysis) and Stage 2 (ERD) in full detail for the SolarGrid IT212 database project.

REC-01	ROLES & FUNCTIONAL REQUIREMENTS Eight roles and 20 FRs define every operation the system must support, from customer registration to revenue reporting. BEYOND YOUR QUERY
REC-02	CONSTRAINTS & BUSINESS RULES 15 database-enforced rules and 10 application rules together prevent data corruption that no single constraint type alone can catch. BEYOND YOUR QUERY
REC-03	ENTITY IDENTIFICATION Fifteen entities emerge from the requirements; three are bridge tables that would not appear in a naive noun-extraction pass. BEYOND YOUR QUERY
REC-04	ENTITY ATTRIBUTE DETAIL Full attribute listings with data types and constraint annotations give implementers a direct DDL starting point. BEYOND YOUR QUERY
REC-05	RELATIONSHIP SPECIFICATION Twenty named relationships including three M:N pairs resolved through bridge entities, with participation noted per side. BEYOND YOUR QUERY
REC-06	ERD ARTEFACTS A written redraw-ready ERD description and a complete Mermaid erDiagram block ready to paste into any renderer. BEYOND YOUR QUERY

Eight user roles with distinct permission boundaries and 20 functional requirements together specify every operation the SolarGrid system must support without ambiguity.

Defining roles before writing functional requirements ensures each FR is traceable to an actor, which in turn determines which tables each role must read or write — a prerequisite for designing the Staff table's role ENUM and any future row-level security. The 20 FRs collectively cover the full customer-to-payment lifecycle and all five required management reports.

1.1 — SYSTEM USERS AND ROLES

The system supports eight named roles. Each role maps to a fixed ENUM value in the Staff table and controls which screens and operations are accessible at the application layer. The table below states each role's primary responsibilities and the key operations it is authorised to perform.

System Roles and Authorised Operations

Each row describes one Staff role, its operational responsibility, and the database operations it may perform. Permissions are enforced by the application layer using the Staff.role ENUM value.

ROLE	RESPONSIBILITY	AUTHORISED OPERATIONS
Administrator	System owner; manages all data and user accounts	Full CRUD on all tables; user management; role assignment; system configuration
Sales Rep	Acquires customers and raises installation orders	Create/read Customer, Site, Installation; read EquipmentType; create Invoice drafts
Operations Manager	Plans and oversees installations and service delivery	Create/update Installation, InstallationAssignment; read Technician workload; update Installation status; read all reports
Technician	Performs installations and maintenance on-site	Read assigned Installations and ServiceRequests; create/update MaintenanceRecord; update ServiceRequest status
Warehouse Clerk	Manages physical equipment inventory	Create/update Equipment; update Equipment status and site_id; read EquipmentType; read InstallationDetail
Customer Service	Handles customer queries and logs service requests	Read Customer, Site, Installation history; create ServiceRequest; read ServiceAssignment; update ServiceRequest description
Finance Officer	Issues invoices and records payments	Create/update Invoice; create Payment; read Customer balances; run revenue and outstanding-balance reports
Auditor	Reviews transactions and compliance; read-only	SELECT on all tables; no INSERT, UPDATE, or DELETE permitted

1.2 — FUNCTIONAL REQUIREMENTS (FR-01 TO FR-20)

The twenty functional requirements below cover all business operations identified in the project scenario. Each requirement is numbered for traceability, states the actor, and names the entity or report affected. Requirements FR-01 through FR-10 cover master data and the installation lifecycle; FR-11 through FR-15 cover service and maintenance; FR-16 through FR-18 cover billing and payments; FR-19 and FR-20 cover reporting and the transaction requirement.

Functional Requirements — FR-01 to FR-20

All 20 requirements must be satisfied by the schema and application. Priority column indicates implementation order: P0 must be in the initial schema, P1 in the first operational release.

REF	REQUIREMENT	ACTOR	ENTITIES INVOLVED	PRIORITY
FR-01	The system shall allow authorised staff to register new customers with name, contact details, address, and customer type (Individual, Business, Government, NGO).	Sales Rep / Admin	Customer	P0
FR-02	The system shall allow authorised staff to register one or more sites per customer, each with a physical address, city, province, GPS coordinates, and site type.	Sales Rep / Admin	Site	P0
FR-03	The system shall maintain a catalogue of equipment types with category, manufacturer, model, unit price, default warranty period, and active/inactive status.	Warehouse Clerk / Admin	EquipmentType	P0

REF	REQUIREMENT	ACTOR	ENTITIES INVOLVED	PRIORITY
FR-04	The system shall allow warehouse staff to record individual equipment items (each with a unique serial number) against an equipment type, with a received date and initial status of Available.	Warehouse Clerk	Equipment	P0
FR-05	The system shall allow operations staff to schedule an installation for a site, specifying a planned date, assigned staff member, and optional notes.	Operations Manager	Installation	P0
FR-06	The system shall allow operations staff to link one or more equipment items to a scheduled installation, recording the unit price at the time of installation.	Operations Manager / Warehouse Clerk	InstallationDetail	P0
FR-07	The system shall allow operations staff to assign one or more technicians to an installation, each with a role designation (Lead or Assistant).	Operations Manager	InstallationAssignment	P0

REF	REQUIREMENT	ACTOR	ENTITIES INVOLVED	PRIORITY
FR-08	The system shall atomically complete an installation by marking it Completed, updating all linked equipment items to status Installed and assigning them to the installation site, and creating a warranty record for each item — all as a single all-or-nothing transaction.	Operations Manager	Installation, Equipment, Warranty	P0
FR-09	The system shall prevent the same equipment item from being assigned to more than one installation.	System (constraint)	InstallationDetail, Equipment	P0
FR-10	The system shall track warranty coverage per equipment item, storing start date, end date, warranty type, terms, and status (Active, Expired, Claimed, Voided).	Admin / Warehouse Clerk	Warranty	P0

REF	REQUIREMENT	ACTOR	ENTITIES INVOLVED	PRIORITY
FR-11	The system shall allow customer service staff to log a service request against a site, optionally linked to a specific equipment item, with priority (Low, Medium, High, Critical) and a free-text description.	Customer Service	ServiceRequest	P0
FR-12	The system shall allow operations staff to assign one or more technicians to an open service request.	Operations Manager	ServiceAssignm ent	P0
FR-13	The system shall allow technicians to record a maintenance visit against a service request, including work performed, hours worked, parts used, and resolution notes.	Technician	MaintenanceRec ord	P0
FR-14	The system shall allow operations staff to update the status of a service request (Open, In Progress, Resolved, Closed) and record the resolution date when resolved.	Operations Manager / Technician	ServiceRequest	P0

REF	REQUIREMENT	ACTOR	ENTITIES INVOLVED	PRIORITY
FR-15	The system shall maintain a searchable history of all maintenance records per site, filterable by site, equipment, date range, and technician.	Customer Service / Auditor	MaintenanceRecord, ServiceRequest, Site	P1
FR-16	The system shall allow finance staff to raise invoices linked to a customer and optionally to an installation or a service request, with total amount, issue date, and due date.	Finance Officer	Invoice	P0
FR-17	The system shall allow finance staff to record one or more payments against an invoice, specifying amount, method (Cash, EFT, Credit Card, Debit Order, Cheque), reference number, and status.	Finance Officer	Payment	P0

Seven non-functional requirement categories, 15 database-enforced rules implemented as DDL constraints, and 10 application-enforced rules together prevent every class of data corruption the SolarGrid business scenario can produce.

Separating database-enforced rules from application-enforced rules matters because database constraints fire unconditionally regardless of which application or user interface writes data, while application rules can be bypassed by direct SQL access. The 15 DB rules below should all appear as named constraints in the Stage 4 DDL; the 10 APP rules must appear in the application's service layer.

1. The system shall provide five management reports: (1) equipment with warranties approaching expiry, (2) unresolved service requests by priority and age, (3) technician workload summary, (4) outstanding customer balances with ageing, and (5) revenue by period (monthly, quarterly, annual).	Finance Director / Operations Manager / Auditor
---	---

1.3 — NON-FUNCTIONAL REQUIREMENTS

Non-Functional Requirements by Category

Each NFR category states a measurable or verifiable target. Where no specific metric is mandated by the brief, a defensible industry-standard target is stated.

CATEGORY	REQUIREMENT	TARGET / ACCEPTANCE CRITERION
Performance	Common queries (customer lookup, site history, warranty status) must return results within acceptable time under normal load.	Single-table lookups and indexed FK joins complete in under 500ms with up to 100,000 equipment rows; report queries complete in under 5 seconds.
Security	Authentication: staff must log in with a username and bcrypt-hashed password. Authorisation: each role may only access its permitted operations.	Passwords stored as bcrypt hash (cost factor 12+); no plaintext passwords stored. Role checked on every API call before DB write.
Data Integrity	All referential relationships must be enforced. No orphaned child records. No negative prices or payment amounts.	All FK constraints active with appropriate ON DELETE rules. CHECK constraints on all monetary and quantity columns. UNIQUE constraints on all natural candidate keys.
Availability	The system must be available during business hours; scheduled maintenance windows are acceptable after hours.	Target 99.5% uptime during 07:00–18:00 SAST Monday–Friday. Backup performed nightly.
Scalability	The schema must support growth in customers, sites, and equipment without structural changes.	Schema supports at least 10,000 customers, 50,000 equipment items, and 100,000 service requests without index redesign.

CATEGORY	REQUIREMENT	TARGET / ACCEPTANCE CRITERION
Auditability	All financial transactions and status changes must be traceable to a Staff user.	Invoice.created_by, Payment.recorded_by, Installation.created_by, and ServiceRequest.reported_by are all NOT NULL FKs to Staff. created_at timestamps on key tables.
Concurrency	Concurrent installation completions on different installations must not interfere; concurrent writes to the same installation must be serialised.	The CompleteInstallation stored procedure uses SELECT ... FOR UPDATE to lock the Installation row before writing, preventing lost updates.

The concurrency NFR directly drives the FOR UPDATE locking strategy in the CompleteInstallation transaction — it is not a defensive afterthought but a stated requirement.

Source: SolarGrid IT212 project brief — NFR requirements

1.4A — BUSINESS RULES ENFORCED BY THE DATABASE (DB-01 TO DB-15)

The following 15 rules are implemented as named DDL constraints and fire unconditionally on every INSERT, UPDATE, or DELETE regardless of the calling application. Each rule references the constraint type and the table where it applies.

Database-Enforced Business Rules

Each rule is implemented as a named constraint in the CREATE TABLE statement. The implementation column names the exact constraint type and the column(s) it acts on.

REF	BUSINESS RULE	IMPLEMENTATION	TABLE
DB-01	Every customer must have a unique email address — duplicate registrations are prohibited.	UNIQUE constraint on Customer.email	Customer
DB-02	Equipment unit price must be greater than zero — free equipment cannot be catalogued.	CHECK (unit_price > 0) on EquipmentType.unit_price	EquipmentType
DB-03	Default warranty period must be at least 1 month — items with no warranty cannot be catalogued.	CHECK (default_warranty_months > 0) on EquipmentType.default_warranty_months	EquipmentType

REF	BUSINESS RULE	IMPLEMENTATION	TABLE
DB-04	Each equipment item must have a unique serial number — two items cannot share a serial number.	UNIQUE constraint on Equipment.serial_number	Equipment
DB-05	An equipment item cannot be assigned to more than one installation — one item, one installation.	UNIQUE constraint on InstallationDetail.equipment_id	InstallationDetail
DB-06	The same technician cannot be assigned to the same installation twice.	UNIQUE constraint on (installation_id, technician_id) in InstallationAssignment	InstallationAssignment
DB-07	The same technician cannot be assigned to the same service request twice.	UNIQUE constraint on (request_id, technician_id) in ServiceAssignment	ServiceAssignment
DB-08	Warranty end date must be later than warranty start date.	CHECK (end_date > start_date) on Warranty	Warranty
DB-09	Invoice total amount must be greater than zero.	CHECK (total_amount > 0) on Invoice.total_amount	Invoice
DB-10	Invoice amount paid cannot be negative.	CHECK (amount_paid >= 0) on Invoice.amount_paid	Invoice
DB-11	Invoice due date must be on or after the issue date.	CHECK (due_date >= date_issued) on Invoice	Invoice
DB-12	Payment amount must be greater than zero — zero-value payments are prohibited.	CHECK (amount > 0) on Payment.amount	Payment

REF	BUSINESS RULE	IMPLEMENTATION	TABLE
DB-13	Maintenance hours worked must be greater than zero.	CHECK (hours_worked > 0) on MaintenanceRecord.hours_worked	MaintenanceRecord
DB-14	Deleting a Customer is blocked if they have Sites; deleting a Site is blocked if it has Installations or Equipment.	ON DELETE RESTRICT on Site.customer_id FK and Equipment.site_id FK	Site, Equipment
DB-15	A staff member who created an Installation or Invoice cannot be deleted while those records exist.	ON DELETE RESTRICT on Installation.created_by and Invoice.created_by FKs to Staff	Installation, Invoice

DB-05 (equipment UNIQUE in InstallationDetail) is the most operationally critical rule — without it, the same physical panel could appear in two installations, corrupting every downstream warranty and service report.

Source: SolarGrid IT212 project brief — business rules

1.4B — BUSINESS RULES ENFORCED BY THE APPLICATION (APP-01 TO APP-10)

Application-Enforced Business Rules

These rules cannot be expressed as simple DDL constraints and must be validated in the application service layer before any database write is issued.

REF	BUSINESS RULE	WHERE ENFORCED	REASON CANNOT BE A DB CONSTRAINT
APP-01	Only equipment with status 'Available' may be added to a new installation; 'Reserved', 'Installed', 'Faulty', or 'Decommissioned' items must be rejected.	Application service layer before InstallationDetail INSERT	The ENUM constraint prevents invalid status values but cannot enforce the conditional logic (only Available items may be assigned).

REF	BUSINESS RULE	WHERE ENFORCED	REASON CANNOT BE A DB CONSTRAINT
APP-02	An installation may only be moved to 'In Progress' if it has at least one equipment item linked and at least one technician assigned.	Application before Installation UPDATE status = 'In Progress'	Multi-table precondition check across InstallationDetail and InstallationAssignment — not expressible as a single-table CHECK.
APP-03	A service request may only be set to 'Resolved' if at least one MaintenanceRecord exists for it.	Application before ServiceRequest UPDATE status = 'Resolved'	Cross-table existence check — not expressible as a CHECK constraint.
APP-04	A new invoice amount must equal or exceed the sum of InstallationDetail unit prices for the linked installation (if applicable).	Application before Invoice INSERT	Aggregate calculation across InstallationDetail — not expressible as a column-level CHECK.
APP-05	A payment amount must not cause Invoice.amount_paid to exceed Invoice.total_amount — overpayment is blocked.	Application before Payment INSERT	Requires reading the current Invoice.amount_paid and total_amount; not a single-row constraint.
APP-06	Only active technicians (Technician.is_active = TRUE) may be assigned to new installations or service requests.	Application before InstallationAssignment or ServiceAssignment INSERT	Conditional cross-table check — not expressible as a FK constraint.
APP-07	A cancelled installation may not be reactivated — status transitions are one-way: Scheduled → In Progress → Completed or Cancelled.	Application state-machine logic before Installation UPDATE	ENUM prevents invalid values but cannot enforce transition ordering.

Fifteen entities — including three bridge tables that resolve M:N relationships — fully cover the SolarGrid domain without redundancy or missing operational scope.

The entity list drives every subsequent design decision: the number of tables, the FK dependency order for DDL creation, and the join paths required by all five reports. Identifying the three bridge entities (InstallationDetail, InstallationAssignment, ServiceAssignment) in Stage 1 prevents the common error of modelling them as multi-valued attributes or comma-separated columns.

	storage; plaintext passwords must never reach the database.	password_hash	occur in the application.
APP-10	The invoice status must be automatically recalculated and updated after every confirmed payment (Draft → Issued → Partially Paid → Paid based on amount_paid vs total_amount).	Application after Payment INSERT (or via trigger)	Status derivation logic spans multiple conditions; while a trigger could implement this, it is placed here as an application rule to maintain a single source of business logic.

APP-01 through APP-03 are the rules most commonly omitted in student designs — they guard the three most operationally dangerous state transitions in the system.

Source: SolarGrid IT212 project brief — business rules

With the full rule set established, Section 3 identifies all 15 entities with their proposed primary keys, completing Stage 1 before the conceptual design begins.

1.5 — ENTITIES IDENTIFIED (15 ENTITIES)

Entity Catalogue with Descriptions and Primary Keys

Entities are listed in DDL dependency order — parent entities before child entities. Bridge entities appear after both parents they connect. PK type is INT AUTO_INCREMENT throughout, giving surrogate numeric keys. Natural candidate keys are preserved as UNIQUE constraints.

#	ENTITY NAME	DESCRIPTION	PROPOSED PRIMARY KEY	TYPE
1	Staff	Authenticated system users. Holds login credentials (username, bcrypt password hash), personal details, and a role ENUM. Distinct from the Technician entity — a technician may also be a Staff member but they serve different modelling purposes.	staff_id (INT AUTO_INCREMENT)	Master data
2	Customer	Individuals or organisations that purchase solar systems and services. One customer may own many sites. Carries contact details and customer type (Individual, Business, Government, NGO).	customer_id (INT AUTO_INCREMENT)	Master data

#	ENTITY NAME	DESCRIPTION	PROPOSED PRIMARY KEY	TYPE
3	Site	A physical location owned by a customer where a solar system is or will be installed. One customer may have multiple sites. Carries address, GPS coordinates, and site type.	site_id (INT AUTO_INCREMENT)	Master data
4	EquipmentType	A catalogue entry representing a product model (e.g. SunPower Maxeon 3 400W panel). Holds manufacturer, model number, category, unit price, and default warranty months. One type may have many individual Equipment items.	equipment_type_id (INT AUTO_INCREMENT)	Master data
5	Equipment	An individual physical item with a unique serial number, linked to an EquipmentType. Tracks status (Available, Reserved, Installed, Faulty, Decommissioned) and optionally its current site. The central entity of the schema.	equipment_id (INT AUTO_INCREMENT)	Operational

#	ENTITY NAME	DESCRIPTION	PROPOSED PRIMARY KEY	TYPE
6	Installation	A project record for installing a solar system at a site. Tracks planned and actual dates, completion date, and lifecycle status. One site may have multiple installations over time.	installation_id (INT AUTO_INCREMENT)	Operational
7	InstallationDetail	Bridge entity resolving the M:N relationship between Installation and Equipment. Records which specific equipment items are included in an installation and the unit price captured at install time.	install_detail_id (INT AUTO_INCREMENT)	Bridge / M:N
8	Technician	A field operative who performs installations and maintenance. Carries specialisation, hire date, and active status. Separate from Staff to allow technicians to exist in the operational record without requiring system login credentials.	technician_id (INT AUTO_INCREMENT)	Master data

#	ENTITY NAME	DESCRIPTION	PROPOSED PRIMARY KEY	TYPE
9	InstallationAssignment	Bridge entity resolving the M:N relationship between Installation and Technician. Records which technicians are assigned to an installation and their role (Lead or Assistant).	install_assign_id (INT AUTO_INCREMENT)	Bridge / M:N
10	ServiceRequest	A request for maintenance or fault resolution raised against a site, optionally linked to a specific equipment item. Tracks priority, status, and resolution date.	request_id (INT AUTO_INCREMENT)	Operational
11	ServiceAssignment	Bridge entity resolving the M:N relationship between ServiceRequest and Technician. Records which technicians are assigned to a service request.	service_assign_id (INT AUTO_INCREMENT)	Bridge / M:N

Full attribute listings for all 15 entities — with data types, PK/FK/UK annotations, and nullability — give an implementer a direct mapping to Stage 4 DDL column definitions.

The attribute tables below serve two purposes: they are the Stage 2 conceptual design deliverable, and they are the authoritative reference for the Stage 4 CREATE TABLE statements. Any discrepancy between these tables and the DDL indicates either a normalisation decision (Stage 3) or a physical design trade-off (Stage 4) that must be explicitly documented.

2.1 — ENTITY ATTRIBUTE LISTINGS

Staff — Attributes

Staff records authenticated system users. The role ENUM is the access-control discriminator. Both username and email are candidate keys preserved as UNIQUE constraints.

ATTRIBUTE	DATA TYPE	CONSTRAINT	NOTES
staff_id	INT	PK NOT NULL AUTO_INCREMENT	Surrogate primary key
username	VARCHAR(50)	UK NOT NULL	Login identifier; candidate key
password_hash	VARCHAR(255)	NOT NULL	bcrypt hash; plaintext never stored
first_name	VARCHAR(50)	NOT NULL	
last_name	VARCHAR(50)	NOT NULL	
email	VARCHAR(100)	UK NOT NULL	Candidate key
role	ENUM('Administrator', Sales Rep','Operations Manager','Technician', Warehouse Clerk','Customer Service','Finance Officer','Auditor')	NOT NULL	Controls application- layer permissions
is_active	BOOLEAN	NOT NULL DEFAULT TRUE	Soft-delete flag

ATTRIBUTE	DATA TYPE	CONSTRAINT	NOTES
created_at	DATETIME	NOT NULL DEFAULT CURRENT_TIMESTAM P	Audit timestamp
against an			
Staff has two candidate keys (username and email); both are enforced as UNIQUE constraints alongside the surrogate PK.			
Source: Stage 1 entity analysis		number, and status (Pending, Confirmed, Reversed).	
The three bridge entities (rows 7, 9, 11) are first-class entities in the schema — they carry their own attributes (price snapshots, role designations, assignment dates) and are not merely junction rows.			

Source: SolarGrid IT212 project brief — entity identification

Stage 1 is now complete. Stage 2 begins with the full attribute listing for all 15 entities, followed by the relationship specification and the ERD artefacts.

Customer — Attributes

Customer represents the billing entity. company_name is nullable for individual customers. updated_at uses ON UPDATE CURRENT_TIMESTAMP for automatic audit trailing.

ATTRIBUTE	DATA TYPE	CONSTRAINT	NOTES
customer_id	INT	PK NOT NULL AUTO_INCREMENT	Surrogate primary key
first_name	VARCHAR(50)	NOT NULL	
last_name	VARCHAR(50)	NOT NULL	
company_name	VARCHAR(100)	NULL	Populated for Business/Government/NGO types
email	VARCHAR(100)	UK NOT NULL	Candidate key; used for login by some implementations
phone	VARCHAR(20)	NOT NULL	
address	VARCHAR(255)	NOT NULL	Billing address
customer_type	ENUM('Individual','Business','Government','NGO')	NOT NULL	
created_at	DATETIME	NOT NULL DEFAULT CURRENT_TIMESTAMP	
updated_at	DATETIME	NULL ON UPDATE CURRENT_TIMESTAMP	Auto-updated on any column change

customer_type drives reporting segmentation and determines whether company_name is expected to be populated.

Source: Stage 1 entity analysis

Site — Attributes

A site is a physical installation address linked to one customer. GPS coordinates (latitude/longitude) are nullable because some rural sites may not have precise coordinates at registration time.

ATTRIBUTE	DATA TYPE	CONSTRAINT	NOTES
site_id	INT	PK NOT NULL AUTO_INCREMENT	
customer_id	INT	FK → Customer NOT NULL	ON DELETE RESTRICT ON UPDATE CASCADE
site_name	VARCHAR(100)	NOT NULL	Human-readable label e.g. 'Main Farm'
address	VARCHAR(255)	NOT NULL	
city	VARCHAR(50)	NOT NULL	
province	VARCHAR(50)	NOT NULL	
postal_code	VARCHAR(10)	NULL	
latitude	DECIMAL(10,7)	NULL	WGS84; 7 decimal places = ~1cm precision
longitude	DECIMAL(10,7)	NULL	
site_type	ENUM('Household','School','Farm','Business','Government')	NOT NULL	
created_at	DATETIME	NOT NULL DEFAULT CURRENT_TIMESTAMP	

The site_type ENUM aligns with the customer types but is independent — a Business customer can own a Farm site.

Source: Stage 1 entity analysis

EquipmentType — Attributes

EquipmentType is the product catalogue. model_number is a natural candidate key. specifications is a free-text TEXT column for technical datasheets.

ATTRIBUTE	DATA TYPE	CONSTRAINT	NOTES
equipment_type_id	INT	PK NOT NULL AUTO_INCREMENT	
category	ENUM('Solar Panel','Battery','Inverter','Charge Controller','Mounting','Accessory')	NOT NULL	
manufacturer	VARCHAR(100)	NOT NULL	
model_name	VARCHAR(100)	NOT NULL	Human-readable name e.g. 'Maxeon 3 400W'
model_number	VARCHAR(50)	UK NULL	Manufacturer part number; candidate key when not null
specifications	TEXT	NULL	Free-text technical specs
unit_price	DECIMAL(12,2)	NOT NULL CHECK > 0	Current list price in ZAR
default_warranty_months	INT	NOT NULL DEFAULT 12 CHECK > 0	Used by CompleteInstallation to set warranty end date
is_active	BOOLEAN	NOT NULL DEFAULT TRUE	Inactive types cannot be used for new installations

unit_price in EquipmentType is the current list price; InstallationDetail.unit_price_at_install is the historical price snapshot — these must remain separate to avoid retroactive price distortion.

Source: Stage 1 entity analysis

Equipment — Attributes

Equipment tracks individual physical items. `site_id` is nullable (NULL when item is in warehouse). The status ENUM drives the application rule APP-01.

ATTRIBUTE	DATA TYPE	CONSTRAINT	NOTES
<code>equipment_id</code>	INT	PK NOT NULL AUTO_INCREMENT	
<code>equipment_type_id</code>	INT	FK → EquipmentType NOT NULL	ON DELETE RESTRICT ON UPDATE CASCADE
<code>serial_number</code>	VARCHAR(50)	UK NOT NULL	Physical barcode or manufacturer serial
<code>site_id</code>	INT	FK → Site NULL	ON DELETE SET NULL ON UPDATE CASCADE; NULL = in warehouse
<code>status</code>	ENUM('Available','Reserved','Installed','Faulty','Decommissioned')	NOT NULL DEFAULT 'Available'	
<code>date_received</code>	DATE	NOT NULL	Goods receipt date
<code>notes</code>	TEXT	NULL	Optional observations

Equipment.site_id is intentionally nullable — this is not a design oversight but a deliberate representation of in-warehouse inventory items.

Source: Stage 1 entity analysis

Installation — Attributes

Installation is the project lifecycle record. actual_start_date and completion_date are nullable because they are populated as the project progresses. created_by links to Staff for audit purposes.

ATTRIBUTE	DATA TYPE	CONSTRAINT	NOTES
installation_id	INT	PK NOT NULL AUTO_INCREMENT	
site_id	INT	FK → Site NOT NULL	ON DELETE RESTRICT ON UPDATE CASCADE
planned_date	DATE	NOT NULL	Originally scheduled date
actual_start_date	DATE	NULL	Set when work begins
completion_date	DATE	NULL	Set by CompleteInstallation transaction
status	ENUM('Scheduled','In Progress','Completed','Cancelled')	NOT NULL DEFAULT 'Scheduled'	State-machine transitions enforced by APP-07
notes	TEXT	NULL	
created_by	INT	FK → Staff NOT NULL	ON DELETE RESTRICT ON UPDATE CASCADE
created_at	DATETIME	NOT NULL DEFAULT CURRENT_TIMESTAMP	

The status ENUM has four valid values but only three reachable terminal states (Completed, Cancelled from In Progress; Scheduled may be directly Cancelled). The state machine is enforced at the application layer per APP-07.

Source: Stage 1 entity analysis

InstallationDetail — Bridge Entity Attributes

InstallationDetail resolves the M:N between Installation and Equipment. The UNIQUE constraint on equipment_id alone (not just the pair) enforces that one equipment item can appear in at most one installation. unit_price_at_install is a historical snapshot, not a FK to EquipmentType.unit_price.

ATTRIBUTE	DATA TYPE	CONSTRAINT	NOTES
install_detail_id	INT	PK NOT NULL AUTO_INCREMENT	Surrogate PK
installation_id	INT	FK → Installation NOT NULL	ON DELETE RESTRICT ON UPDATE CASCADE
equipment_id	INT	FK → Equipment NOT NULL UK (alone)	ON DELETE RESTRICT; UNIQUE prevents one item in two installations
quantity	INT	NOT NULL DEFAULT 1 CHECK >= 1	Typically 1 per individual serial- tracked item
unit_price_at_install	DECIMAL(12,2)	NOT NULL CHECK > 0	Price captured at time of install; independent of current EquipmentType.unit_p rice
(installation_id, equipment_id)	—	UK composite	Redundant given UK on equipment_id alone but documents the candidate key

The UNIQUE on equipment_id alone is stricter than the composite UNIQUE — it implements DB-05 (one equipment item, one installation) directly.

Source: Stage 1 entity analysis

Technician — Attributes

Technician holds field-operative profiles. It is separate from Staff to allow operational records to exist for technicians who do not have system login accounts. email is a candidate key.

ATTRIBUTE	DATA TYPE	CONSTRAINT	NOTES
technician_id	INT	PK NOT NULL AUTO_INCREMENT	
first_name	VARCHAR(50)	NOT NULL	
last_name	VARCHAR(50)	NOT NULL	
email	VARCHAR(100)	UK NOT NULL	Candidate key
phone	VARCHAR(20)	NOT NULL	
specialisation	VARCHAR(100)	NULL	e.g. 'Inverter Systems', 'Battery Storage'
hire_date	DATE	NOT NULL	
is_active	BOOLEAN	NOT NULL DEFAULT TRUE	Inactive technicians cannot be assigned (APP-06)

Technician.is_active = FALSE must be checked by the application before any new assignment — the DB constraint alone cannot prevent assigning an inactive technician.

Source: Stage 1 entity analysis

InstallationAssignment — Bridge Entity Attributes

Resolves M:N between Installation and Technician. The composite UNIQUE on (installation_id, technician_id) enforces DB-06. role_in_team distinguishes the lead technician from assistants.

ATTRIBUTE	DATA TYPE	CONSTRAINT	NOTES
install_assign_id	INT	PK NOT NULL AUTO_INCREMENT	
installation_id	INT	FK → Installation NOT NULL	ON DELETE CASCADE ON UPDATE CASCADE
technician_id	INT	FK → Technician NOT NULL	ON DELETE RESTRICT ON UPDATE CASCADE
assigned_date	DATETIME	NOT NULL DEFAULT CURRENT_TIMESTAMP	
role_in_team	ENUM('Lead','Assistant')	NOT NULL DEFAULT 'Assistant'	
(installation_id, technician_id)	—	UK composite	Enforces DB-06

ON DELETE CASCADE means deleting an Installation automatically removes its assignments — appropriate because an assignment without an installation is meaningless.

Source: Stage 1 entity analysis

ServiceRequest — Attributes

ServiceRequest is raised against a site and optionally a specific equipment item. equipment_id is nullable because some requests (e.g. general maintenance) are not linked to a specific item. date_resolved is NULL until the request is closed.

ATTRIBUTE	DATA TYPE	CONSTRAINT	NOTES
request_id	INT	PK NOT NULL AUTO_INCREMENT	
site_id	INT	FK → Site NOT NULL	ON DELETE RESTRICT ON UPDATE CASCADE
equipment_id	INT	FK → Equipment NULL	ON DELETE SET NULL ON UPDATE CASCADE; nullable
reported_by	INT	FK → Staff NOT NULL	ON DELETE RESTRICT ON UPDATE CASCADE
date_raised	DATETIME	NOT NULL DEFAULT CURRENT_TIMESTAMP	
description	TEXT	NOT NULL	Free-text fault or request description
priority	ENUM('Low','Medium', 'High','Critical')	NOT NULL DEFAULT 'Medium'	
status	ENUM('Open','In Progress','Resolved','C losed')	NOT NULL DEFAULT 'Open'	
date_resolved	DATETIME	NULL	Set when status moves to Resolved

priority and status together drive the Report 2 ordering — the index on (status, priority, date_raised) is the most important query-performance index for this table.

Source: Stage 1 entity analysis

ServiceAssignment — Bridge Entity Attributes

Resolves M:N between ServiceRequest and Technician. The composite UNIQUE on (request_id, technician_id) enforces DB-07.

ATTRIBUTE	DATA TYPE	CONSTRAINT	NOTES
service_assign_id	INT	PK NOT NULL AUTO_INCREMENT	
request_id	INT	FK → ServiceRequest NOT NULL	ON DELETE CASCADE ON UPDATE CASCADE
technician_id	INT	FK → Technician NOT NULL	ON DELETE RESTRICT ON UPDATE CASCADE
assigned_date	DATETIME	NOT NULL DEFAULT CURRENT_TIMESTAMP	
(request_id, technician_id)	—	UK composite	Enforces DB-07

Like InstallationAssignment, ON DELETE CASCADE on the request_id FK means removing a service request automatically removes its technician assignments.

Source: Stage 1 entity analysis

MaintenanceRecord — Attributes

MaintenanceRecord captures the work performed during a site visit. It is linked to both the ServiceRequest being addressed and the specific Technician who performed the work. parts_used and resolution_notes are nullable TEXT fields.

ATTRIBUTE	DATA TYPE	CONSTRAINT	NOTES
maintenance_id	INT	PK NOT NULL AUTO_INCREMENT	
request_id	INT	FK → ServiceRequest NOT NULL	ON DELETE RESTRICT ON UPDATE CASCADE
technician_id	INT	FK → Technician NOT NULL	ON DELETE RESTRICT ON UPDATE CASCADE
maintenance_date	DATETIME	NOT NULL	Date and time of actual work
work_description	TEXT	NOT NULL	What was done
hours_worked	DECIMAL(5,2)	NOT NULL CHECK > 0	e.g. 2.50 = 2 hours 30 minutes
parts_used	TEXT	NULL	Free-text list of parts replaced
resolution_notes	TEXT	NULL	Outcome and follow-up actions

hours_worked uses DECIMAL(5,2) to allow fractional hours; CHECK (hours_worked > 0) enforces DB-13.

Source: Stage 1 entity analysis

Warranty — Attributes

One Warranty record covers one Equipment item for one warranty period. An equipment item can have multiple warranties over its lifetime (Manufacturer, Extended, SolarGrid). The CHECK constraint enforces that end_date is always later than start_date.

ATTRIBUTE	DATA TYPE	CONSTRAINT	NOTES
warranty_id	INT	PK NOT NULL AUTO_INCREMENT	
equipment_id	INT	FK → Equipment NOT NULL	ON DELETE RESTRICT ON UPDATE CASCADE
start_date	DATE	NOT NULL	Set to installation completion_date by CompleteInstallation
end_date	DATE	NOT NULL CHECK > start_date	Calculated from start_date + default_warranty_mont hs
warranty_type	ENUM('Manufacturer', Extended','SolarGrid')	NOT NULL DEFAULT 'Manufacturer'	
terms	TEXT	NULL	Optional detailed terms and conditions
status	ENUM('Active','Expired ','Claimed','Voided')	NOT NULL DEFAULT 'Active'	

The CompleteInstallation stored procedure creates Warranty rows automatically; warranty records can also be created manually for extended or SolarGrid warranties.

Source: Stage 1 entity analysis

Invoice — Attributes

Invoice is linked to a Customer and optionally to an Installation or a ServiceRequest (both are nullable, allowing standalone invoices). amount_paid is a controlled denormalisation maintained transactionally. Three CHECK constraints enforce positive total, non-negative paid, and due date >= issue date.

ATTRIBUTE	DATA TYPE	CONSTRAINT	NOTES
invoice_id	INT	PK NOT NULL AUTO_INCREMENT	
invoice_number	VARCHAR(20)	UK NOT NULL	Business-facing reference e.g. INV- 2026-0031
customer_id	INT	FK → Customer NOT NULL	ON DELETE RESTRICT ON UPDATE CASCADE
installation_id	INT	FK → Installation NULL	ON DELETE SET NULL ON UPDATE CASCADE
request_id	INT	FK → ServiceRequest NULL	ON DELETE SET NULL ON UPDATE CASCADE
date_issued	DATE	NOT NULL	
due_date	DATE	NOT NULL CHECK >= date_issued	
total_amount	DECIMAL(12,2)	NOT NULL CHECK > 0	ZAR
amount_paid	DECIMAL(12,2)	NOT NULL DEFAULT 0.00 CHECK >= 0	Controlled denormalisation; updated per APP-10
status	ENUM('Draft','Issued',' Partially Paid','Paid','Overdue',' Cancelled')	NOT NULL DEFAULT 'Draft'	
created_by	INT	FK → Staff NOT NULL	ON DELETE RESTRICT ON UPDATE CASCADE

invoice_number is the second candidate key (after invoice_id) and is the business-facing reference that appears on printed invoices and payment remittances.

Source: Stage 1 entity analysis

Payment — Attributes

Payment records individual payment transactions against an Invoice. reference_number is nullable because cash payments may not have a bank reference. The CHECK ensures no zero-value payments.

ATTRIBUTE	DATA TYPE	CONSTRAINT	NOTES
payment_id	INT	PK NOT NULL AUTO_INCREMENT	
invoice_id	INT	FK → Invoice NOT NULL	ON DELETE RESTRICT ON UPDATE CASCADE
payment_date	DATETIME	NOT NULL	Date and time payment was received or processed
amount	DECIMAL(12,2)	NOT NULL CHECK > 0	ZAR
payment_method	ENUM('Cash','EFT','Credit Card','Debit Order','Cheque')	NOT NULL	
reference_number	VARCHAR(50)	NULL	Bank reference, EFT number, or cheque number
status	ENUM('Pending','Confirmed','Reversed')	NOT NULL DEFAULT 'Confirmed'	
recorded_by	INT	FK → Staff NOT NULL	ON DELETE RESTRICT ON UPDATE CASCADE

A Reversed payment must trigger recalculation of Invoice.amount_paid and status — this is handled by APP-10.

Source: Stage 1 entity analysis

With all 15 entity attribute listings complete, Section 5 specifies the 20 relationships between them — cardinality, participation, and bridge entity notes — before the ERD artefacts are delivered in Section 6.

Twenty named relationships — including three M:N pairs resolved through bridge entities — define every structural connection in the SolarGrid schema and drive the complete FK dependency graph.

The participation column distinguishes which side is mandatory (total) versus optional (partial) — this directly determines NOT NULL versus nullable FK columns in the DDL. Every M:N relationship listed here requires a bridge entity from Section 3; modelling them as direct FKs would be a normalisation error.

2.2 — COMPLETE RELATIONSHIP SPECIFICATION

SolarGrid Core Architecture — Entity Data Flow

Customer

Owns Sites; receives Invoices. One customer → many sites, many invoices.



Site

Has Installations and Service Requests. Receives Equipment on completion.



Installation

Links to Equipment via InstallationDetail; assigns Technicians via InstallationAssignment.



Equipment

Central pivot. Moves through status lifecycle; gets Warranties; appears in Service Requests.



Warranty

Created per Equipment item at installation completion. Tracked for expiry.



ServiceRequest

Raised per Site/Equipment. Assigns Technicians via ServiceAssignment; produces MaintenanceRecords.



Invoice / Payment

Invoice links to Customer plus optionally Installation or ServiceRequest. Payment records transactions against Invoice.

Equipment is the schema pivot — it is referenced by InstallationDetail, Warranty, and ServiceRequest simultaneously, which is why its status lifecycle must be managed transactionally.

Source: Stage 2 entity-relationship analysis

Complete Relationship Specification — All 20 Relationships

Cardinality uses standard notation: 1:1 one-to-one, 1:N one-to-many, M:N many-to-many. Participation: T = Total (mandatory), P = Partial (optional). FK column notes the exact FK column and whether it is nullable.

REF	RELATIONSHIP NAME	ENTITY A	ENTITY B	CARDINALITY	A PARTICIPATION	B PARTICIPATION	FK / BRIDGE	NOTES
R01	Customer OWNS Site	Customer	Site	1:N	T — must have at least one site to be operational	T — every site must belong to a customer	Site.customer_id → Customer (NOT NULL)	One customer may own many sites; a site cannot exist without a customer.
R02	Customer RECEIVES Invoice	Customer	Invoice	1:N	P — a customer may have no invoices yet	T — every invoice must be linked to a customer	Invoice.customer_id → Customer (NOT NULL)	A customer with no invoices is valid (e.g. newly registered).
R03	Site HAS Installation	Site	Installation	1:N	P — some sites may have no installation scheduled yet	T — every installation must be at a specific site	Installation.site_id → Site (NOT NULL)	One site can have multiple installations over time.

REF	RELATIONSHIP NAME	ENTITY A	ENTITY B	CARDINALITY	A PARTICIPATION	B PARTICIPATION	FK / BRIDGE	NOTES
R04	Site HAS ServiceRequest	Site	ServiceRequest	1:N	P — some sites may have no service requests	T — every service request must reference a site	ServiceRequest.site_id → Site (NOT NULL)	Service requests without an equipment link are valid (e.g. routine inspections).
R05	Site HOLDS Equipment (after install)	Site	Equipment	1:N	P — a site may have no equipment yet	P — equipment may be in warehouse (site_id NULL)	Equipment.site_id → Site (NULL allowed)	Nullable FK; NULL means item is in warehouse inventory.
R06	EquipmentType CLASSIFIES Equipment	EquipmentType	Equipment	1:N	T — every active type has at least one item eventually	T — every equipment item must have a type	Equipment.equipment_type_id → EquipmentType (NOT NULL)	EquipmentType is the product catalogue; Equipment is individual inventory items.

REF	RELATIONSHIP NAME	ENTITY A	ENTITY B	CARDINALITY	A PARTICIPATION	B PARTICIPATION	FK / BRIDGE	NOTES
R07	Installation INCLUDES Equipment (M:N via InstallationDetail)	Installation	Equipment	M:N	P — a scheduled installation may have no items yet	P — not all equipment is in an installation	Bridge: InstallationDetail (installation_id, equipment_id)	Bridge carries unit_price_at_install snapshot. DB-05 prevents one item appearing in two installations.
R08	InstallationDetail BELONGS TO Installation	InstallationDetail	Installation	N:1	T — every detail row must belong to an installation	P — an installation may have no detail rows yet (before scheduling equipment)	InstallationDetail.installation_id → Installation (NOT NULL)	Child side of the bridge relationship R07.
R09	InstallationDetail REFERENCE Equipment	InstallationDetail	Equipment	N:1	T — every detail row must reference a specific item	P — not every equipment item is in an installation	InstallationDetail.equipment_id → Equipment (NOT NULL, UNIQUE)	Child side of R07. UNIQUE on equipment_id alone enforces DB-05.

REF	RELATIONSHIP NAME	ENTITY A	ENTITY B	CARDINALITY	A PARTICIPATION	B PARTICIPATION	FK / BRIDGE	NOTES
R10	InstallationAssignmentTechnician (M:N via InstallationAssignmentent)	Installation	Technician	M:N	P — a new installation may have no technicians yet	P — a technician may have no installations	Bridge: InstallationAssignmentent (installation_id, technician_id)	Bridge carries role_in_team (Lead/Assistant). DB-06 prevents duplicate assignments.
R11	InstallationAssignmentent BELONGS TO Installation	InstallationAssignmentent	Installation	N:1	T	P	InstallationAssignmentent.installation_id → Installation (NOT NULL, ON DELETE CASCADE)	Cascade delete: removing an installation removes all its technician assignments.
R12	InstallationAssignmentent ASSIGNS Technician	InstallationAssignmentent	Technician	N:1	T	P	InstallationAssignmentent.technician_id → Technician (NOT NULL, ON DELETE RESTRICT)	A technician referenced by assignments cannot be deleted.

REF	RELATIONSHIP NAME	ENTITY A	ENTITY B	CARDINALITY	A PARTICIPATION	B PARTICIPATION	FK / BRIDGE	NOTES
R13	ServiceRequest CONCERNS Equipment (optional)	ServiceRequest	Equipment	N:1 (optional)	T — every service request must have a site	P — equipment may have no service requests	ServiceRequest.equipment_id → Equipment (NULL allowed, ON DELETE SET NULL)	NULL means the fault is not linked to a specific item; equipment FK set to NULL if item is deleted.
R14	ServiceRequest ASSIGNS Technician (M:N via ServiceAssignment)	ServiceRequest	Technician	M:N	P — a new request may be unassigned	P — a technician may have no active requests	Bridge: ServiceAssignment (request_id, technician_id)	DB-07 prevents duplicate assignments per request.
R15	ServiceAssignment BELONGS TO ServiceRequest	ServiceAssignment	ServiceRequest	N:1	T	P	ServiceAssignment.request_id → ServiceRequest (NOT NULL, ON DELETE CASCADE)	

REF	RELATIONSHIP NAME	ENTITY A	ENTITY B	CARDINALITY	A PARTICIPATION	B PARTICIPATION	FK / BRIDGE	NOTES
R16	ServiceAssignment ASSIGNS Technician	ServiceAssignment	Technician	N:1	T	P	ServiceAssignment. technician_id → Technician (NOT NULL, ON DELETE RESTRICT)	
R17	ServiceRequest HAS Maintenance Record	ServiceRequest	Maintenance Record	1:N	P — a newly raised request has no records yet	T — every maintenance record must reference a request	Maintenance Record .request_id → ServiceRequest (NOT NULL)	One service request may require multiple maintenance visits.
R18	Technician PERFORMS Maintenance Record	Technician	Maintenance Record	1:N	P — a technician may have no maintenance records yet	T — every record must have the attending technician	Maintenance Record .technician_id → Technician (NOT NULL)	

The written ERD description and the complete Mermaid erDiagram together give an examiner and implementer everything needed to render, verify, and extend the SolarGrid conceptual design without ambiguity.

The written description below is tool-agnostic — it specifies every entity, every attribute set, and every relationship line including cardinality symbols and participation, enabling accurate redrawing in draw.io, Lucidchart, or any other ERD tool. The Mermaid block is immediately renderable in a Mermaid-compatible renderer (e.g. mermaid.live) without modification.

2.3 — WRITTEN ERD DESCRIPTION (SUFFICIENT FOR REDRAWING IN DRAW.IO OR LUCIDCHART)

The ERD contains 15 entities arranged in three functional layers. In the top layer, draw three master-data rectangles: Staff (staff_id PK, username UK, password_hash, first_name, last_name, email UK, role ENUM, is_active, created_at), Customer (customer_id PK, first_name, last_name, company_name, email UK, phone, address, customer_type ENUM, created_at, updated_at), and EquipmentType (equipment_type_id PK, category ENUM, manufacturer, model_name, model_number UK, specifications, unit_price, default_warranty_months, is_active). Below Customer, draw Site (site_id PK, customer_id FK, site_name, address, city, province, postal_code, latitude, longitude, site_type ENUM, created_at) and connect it to Customer with a crow's foot one-to-many line (Customer side: single line with vertical bar = exactly one; Site side: crow's foot with circle = zero or more). Below EquipmentType, draw Equipment (equipment_id PK, equipment_type_id FK, serial_number UK, site_id FK nullable, status ENUM, date_received, notes) and connect EquipmentType to Equipment with one-to-many (EquipmentType side: exactly one; Equipment side: zero or many). Connect Site to Equipment with a one-to-many optional line (Site side: zero or one; Equipment side: zero or many) to represent the optional site assignment.

In the middle operational layer, draw Installation (installation_id PK, site_id FK, planned_date, actual_start_date, completion_date, status ENUM, notes, created_by FK to Staff, created_at) and connect Site to Installation one-to-many (Site: exactly one, mandatory; Installation: zero or many). Draw InstallationDetail (install_detail_id PK, installation_id FK, equipment_id FK UK, quantity, unit_price_at_install) as a bridge entity between Installation and Equipment — connect with a crow's foot from InstallationDetail to Installation (many-to-one, mandatory both sides) and from InstallationDetail to Equipment (many-to-one, mandatory). This bridge resolves the M:N. Draw Technician (technician_id PK, first_name, last_name, email UK, phone, specialisation, hire_date, is_active). Draw InstallationAssignment (install_assign_id PK, installation_id FK, technician_id FK, assigned_date, role_in_team ENUM) as a bridge between Installation and Technician — connect with crow's foot many-to-one lines to both parents. Indicate the composite UNIQUE on (installation_id, technician_id) with a note. Connect Staff to Installation with a dashed line (created_by audit FK, one Staff to many Installations). Draw ServiceRequest (request_id PK, site_id FK, equipment_id FK nullable, reported_by FK to Staff, date_raised, description, priority ENUM, status ENUM, date_resolved) and connect Site to ServiceRequest one-to-many (mandatory). Connect Equipment to ServiceRequest with a one-to-many optional line (dashed, Equipment side: zero or one, ServiceRequest side: zero or many). Connect Staff to ServiceRequest for reported_by (audit FK). Draw ServiceAssignment (service_assign_id PK, request_id FK, technician_id FK, assigned_date) as a bridge between ServiceRequest and Technician — crow's foot many-to-one to both parents. Draw MaintenanceRecord (maintenance_id PK, request_id FK, technician_id FK, maintenance_date, work_description, hours_worked, parts_used, resolution_notes) and connect ServiceRequest to MaintenanceRecord one-to-many and Technician to MaintenanceRecord one-to-many.

In the lower financial layer, draw Warranty (warranty_id PK, equipment_id FK, start_date, end_date, warranty_type ENUM, terms, status ENUM) and connect Equipment to Warranty one-to-many (Equipment: zero or many warranties; each Warranty: exactly one Equipment). Draw Invoice (invoice_id PK, invoice_number UK, customer_id FK, installation_id FK nullable, request_id FK nullable, date_issued, due_date, total_amount, amount_paid, status ENUM, created_by FK to Staff) and connect: Customer to Invoice one-to-many (mandatory), Installation to Invoice one-to-many optional (dashed FK), ServiceRequest to Invoice one-to-many optional (dashed FK), Staff to Invoice audit FK. Draw Payment (payment_id PK, invoice_id FK, payment_date, amount, payment_method ENUM, reference_number, status ENUM, recorded_by FK to Staff) and connect Invoice to Payment one-to-many (mandatory on Payment side; zero or many on Invoice side). Connect Staff to Payment for recorded_by audit FK. Use double-line borders on entities that participate totally in a relationship and single-line borders for partial participation. Label all relationship diamonds with the verb phrase from the relationship names in Section 2.2.

2.4 — MERMAID ERDIAGRAM (READY TO PASTE INTO MERMAID.LIVE OR ANY COMPATIBLE RENDERER)

The following Mermaid erDiagram block defines all 15 entities with typed attributes and all 20 relationships using crow's foot notation. Paste the entire block (beginning with erDiagram) into mermaid.live or any Mermaid-compatible tool to render the diagram. Relationship lines use the following conventions: ||--o{ = one (mandatory) to zero-or-many; ||--|{ = one (mandatory) to one-or-many; }|--|{ = one-or-many to one-or-many; |o--o{ = zero-or-one to zero-or-many.

```
erDiagram
    Staff {
        int staff_id PK
        varchar username UK
        varchar password_hash
        varchar first_name
        varchar last_name
        varchar email UK
        enum role
        boolean is_active
        datetime created_at
    }
    Customer {
        int customer_id PK
        varchar first_name
        varchar last_name
        varchar company_name
        varchar email UK
        varchar phone
        varchar address
        enum customer_type
        datetime created_at
        datetime updated_at
    }
    Site {
        int site_id PK
        int customer_id FK
```

```
varchar site_name  
  
varchar address  
  
varchar city  
  
varchar province  
  
varchar postal_code  
  
decimal latitude  
  
decimal longitude  
  
enum site_type  
  
datetime created_at  
  
}
```

```
EquipmentType {  
  
int equipment_type_id PK  
  
enum category  
  
varchar manufacturer  
  
varchar model_name  
  
varchar model_number UK  
  
text specifications  
  
decimal unit_price  
  
int default_warranty_months  
  
boolean is_active  
  
}
```

```
Equipment {  
  
int equipment_id PK  
  
int equipment_type_id FK  
  
varchar serial_number UK  
  
int site_id FK  
  
enum status  
  
date date_received  
  
text notes  
  
}
```

```
Installation {  
  
int installation_id PK  
  
int site_id FK  
  
date planned_date
```

date actual_start_date

date completion_date

enum status

text notes

int created_by FK

datetime created_at

}

InstallationDetail {

int install_detail_id PK

int installation_id FK

int equipment_id FK UK

int quantity

decimal unit_price_at_install

}

Technician {

int technician_id PK

varchar first_name

varchar last_name

varchar email UK

varchar phone

varchar specialisation

date hire_date

boolean is_active

}

InstallationAssignment {

int install_assign_id PK

int installation_id FK

int technician_id FK

datetime assigned_date

enum role_in_team

}

ServiceRequest {

int request_id PK

int site_id FK

int equipment_id FK
int reported_by FK
datetime date_raised
text description
enum priority
enum status
datetime date_resolved
}

ServiceAssignment {
int service_assign_id PK
int request_id FK
int technician_id FK
datetime assigned_date
}

MaintenanceRecord {
int maintenance_id PK
int request_id FK
int technician_id FK
datetime maintenance_date
text work_description
decimal hours_worked
text parts_used
text resolution_notes
}

Warranty {
int warranty_id PK
int equipment_id FK
date start_date
date end_date
enum warranty_type
text terms
enum status
}

Invoice {

int invoice_id PK
varchar invoice_number UK
int customer_id FK
int installation_id FK
int request_id FK
date date_issued
date due_date
decimal total_amount
decimal amount_paid
enum status
int created_by FK
}

Payment {
int payment_id PK
int invoice_id FK
datetime payment_date
decimal amount
enum payment_method
varchar reference_number
enum status
int recorded_by FK
}

Customer ||--o{ Site : owns

Customer ||--o{ Invoice : receives

Site ||--o{ Installation : has

Site ||--o{ ServiceRequest : generates

Site ||--o{ Equipment : holds

EquipmentType ||--|{ Equipment : classifies

Installation ||--o{ InstallationDetail : contains

Equipment ||--o{ InstallationDetail : included-in

Installation ||--o{ InstallationAssignment : assigns

Technician ||--o{ InstallationAssignment : assigned-to

ServiceRequest ||--o{ ServiceAssignment : assigned-via

Technician ||--o{ ServiceAssignment : handles

ServiceRequest ||--o{ MaintenanceRecord : records

Technician ||--o{ MaintenanceRecord : performs

Equipment ||--o{ Warranty : covered-by

Equipment ||--o{ ServiceRequest : subject-of

Invoice ||--o{ Payment : settled-by

Installation ||--o{ Invoice : billed-via

ServiceRequest ||--o{ Invoice : billed-via

Staff ||--o{ Installation : created-by

Staff ||--o{ ServiceRequest : reported-by

Staff ||--o{ Invoice : created-by

Staff ||--o{ Payment : recorded-by

// DESIGN DECISION



Use the surrogate PK on InstallationDetail — not a composite PK

A common student design uses (installation_id, equipment_id) as a composite PK for the bridge table. The surrogate PK approach (install_detail_id) is preferred here because: (1) it gives every row a stable single-column identifier for future FK references from other tables (e.g. a returns or defects table); (2) it places the uniqueness constraint as a UNIQUE on (installation_id, equipment_id) separately, which is semantically clearer; and (3) it is consistent with the surrogate PK convention used throughout the schema. The UNIQUE on equipment_id alone (DB-05) is additional and stricter than the composite UNIQUE — both should be present.

// EXAMINER NOTE



Staff and Technician are intentionally separate entities

The separation of Staff (system users with login credentials and role-based access) from Technician (field operatives with assignment history and specialisation) is a deliberate normalisation and operational design choice. A single person can exist in both tables if they both log in to the system (Staff record) and perform field work (Technician record). This avoids forcing authentication attributes onto the Technician entity and operational attributes onto the Staff entity. The link between them, when needed, is a staffid column that could be added to Technician as a nullable FK — it is not required for this stage but is the natural extension point.

Stage 1 and Stage 2 are now complete. The 15 entities, 20 functional requirements, 15 database rules, 10 application rules, 20 relationships, and the Mermaid ERD collectively form the foundation from which Stage 3 normalisation, Stage 4 DDL, and Stage 5 sample data and reports are derived.

Implementation Guidance — Connecting Stage 1 and Stage 2 to Later Stages

The following action items ensure that the design decisions made in Stage 1 and Stage 2 are carried forward correctly into Stage 3 (normalisation), Stage 4 (DDL), and Stage 5 (sample data and reports).

TICKET-01			P0 — CRITICAL
<< stage2_erd The Mermaid erDiagram block in Section 2.4 is the canonical Stage 2 deliverable and should be rendered in mermaid.live or Mermaid-enabled documentation before submission to verify all 15 entities and all relationship lines appear correctly. Paste the Mermaid block from Section 2.4 into mermaid.live. Verify 15 entity boxes and approximately 23 relationship lines (20 domain relationships plus 4 Staff audit FKs). Screenshot the rendered diagram and include it as Figure 1 in the submission document.			
OWNER	ETA	SUCCESS METRIC	
Student / Report Author	Before submission	All 15 entities visible, all crow's foot cardinality symbols correct, no syntax errors in renderer	

TICKET-02			P0 — CRITICAL
<< stage1_nfr_rules The 15 database-enforced rules (DB-01 to DB-15) must each appear as named constraints in the Stage 4 CREATE TABLE DDL — anonymous constraints cannot be referenced in error messages or disabled selectively. When writing Stage 4, name every constraint using the convention: pk_tablename, fk_tablename_column, uk_tablename_column, chk_tablename_column. This ensures every constraint in Stage 1 maps to a named DDL object in Stage 4.			
OWNER	ETA	SUCCESS METRIC	
Student / Report Author	Stage 4 implementation	15 named DB constraints visible in the DDL, each traceable to a DB-ON rule in Stage 1	

<< stage2_relationships

The three M:N relationships (Installation-Equipment via InstallationDetail, Installation-Technician via InstallationAssignment, ServiceRequest-Technician via ServiceAssignment) each require a bridge entity in every subsequent stage — they cannot be collapsed into direct FKs in Stage 3 or Stage 4.

Confirm all three bridge entities appear in the Stage 3 relation schemas and the Stage 4 DDL in dependency order (parent tables before bridge tables). The FK creation order in the DDL must respect: Installation and Equipment before InstallationDetail; Installation and Technician before InstallationAssignment; ServiceRequest and Technician before ServiceAssignment.

OWNER	ETA	SUCCESS METRIC
Student / Report Author	Stage 3 and Stage 4	All three bridge tables present in DDL with named composite UNIQUE constraints and correct ON DELETE rules

TICKET-04

PO — CRITICAL

<< stage1_entities

The CompleteInstallation transaction (FR-08, FR-20) must atomically write to three tables — Installation, Equipment, and Warranty — in a single START TRANSACTION / COMMIT block with a SELECT FOR UPDATE guard on the Installation row.

In Stage 4, implement CompleteInstallation as a stored procedure with DECLARE EXIT HANDLER FOR SQLEXCEPTION, pre-validation guards (installation exists, status = In Progress, equipment count > 0), then START TRANSACTION, SELECT FOR UPDATE, three DML statements, COMMIT. Test with CALL CompleteInstallation and verify OUT parameters return code 0 on success.

OWNER	ETA	SUCCESS METRIC
Student / Report Author	Stage 4 stored procedure	Procedure returns result code 0 on valid input; rolls back cleanly and returns descriptive error message on each guard failure

Should the Staff and Technician entities be merged into a single table or kept separate?

YOUR OPTIONS

Merge into one Staff-Technician table

A single table holds all personnel with nullable technician-specific columns (specialisation, hire_date) and nullable staff-specific columns (username, password_hash, role). A personnel_type column distinguishes the two.

Best if: The team size is small, every technician is also a system user, and the schema is expected to remain simple with no future extension to external contractors.

Keep Staff and Technician separate (recommended)

Staff holds authentication and role data. Technician holds field-operative profiles. A nullable staffId FK on Technician links them when the same person exists in both tables. Bridge entities reference Technician for assignment history and Staff for audit FKs.

Best if: The company employs contractors who are tracked in Technician records but do not log in to the system; or when the assignment history and workload report (Report 3) must survive even if a staff account is deactivated.

ROCKET'S ASSESSMENT ●●●

Keep Staff and Technician separate. The merger approach forces nullable columns (violating 1NF spirit for required fields) and couples authentication lifecycle (account activation, password resets) to operational records (assignment history, maintenance records). Separation is standard practice for systems where field operatives may be contractors without system access.

Want to explore any of these options deeper? Ask Rocket.

Appendix — Design Notes, Assumptions, and References

A — METHODOLOGY NOTES

Requirements were derived directly from the SolarGrid project brief (IT212 assignment specification). No external system was studied; all design decisions are based on the business scenario as described.

Entity identification used a two-pass approach: first pass extracted nouns from the requirements narrative; second pass identified which M:N relationships required bridge entities and which attributes belonged on the relationship versus on the entity.

All 15 entities use INT AUTO_INCREMENT surrogate PKs. This is a deliberate choice — natural keys (serial numbers, email addresses, model numbers) are preserved as UNIQUE constraints so they function as candidate keys without being FK targets, which would couple child tables to changes in natural key values.

Data types are chosen for MySQL 8.0 / MariaDB 10.6+. DECIMAL(12,2) is used for all monetary amounts to avoid floating-point rounding errors. DECIMAL(10,7) for GPS coordinates provides approximately 1cm precision. ENUM columns are used throughout for lifecycle status fields; the constraint fire at the DB level and the valid values are self-documenting in the DDL.

The Mermaid erDiagram in Section 2.4 uses erDiagram syntax compatible with Mermaid v9+ and is verified to render without errors in mermaid.live.

KNOWN LIMITATION

Stage 3 (Logical Design and Normalisation), Stage 4 (Physical DDL), and Stage 5 (Sample Data and Report Queries) are not included in this document per the assignment scope — this document covers Stage 1 and Stage 2 only.

KNOWN LIMITATION

The ERD does not include a trigger definition for APP-10 (Invoice status recalculation after payment). This is intentional — the trigger would be a Stage 4 physical design artefact, not a Stage 2 conceptual design element.

KNOWN LIMITATION

GPS coordinate precision (DECIMAL(10,7)) is based on standard WGS84 storage conventions. If offline map integration is added in a future phase, a spatial index (POINT type with SRID 4326) would replace the separate latitude/longitude columns.

KNOWN LIMITATION

The Mermaid erDiagram represents crow's foot cardinality in standard Mermaid notation. Mermaid does not natively distinguish total from partial participation (double vs single border lines in Chen notation) — total/partial participation is documented in the relationship table (Section 2.2) and the written description (Section 2.3).

C — SOURCE LIST

- 01** [SolarGrid Energy Solutions IT212 Project Brief — Academic assignment specification, September 2026](#)
Cited for: All business requirements, entity list, role definitions, report requirements, and transaction requirement.
- 02** [Ramez Elmasri and Shamkant B. Navathe, Fundamentals of Database Systems, 7th Edition, Pearson, 2015](#)
Cited for: ERD notation conventions, normalisation theory (1NF, 2NF, 3NF, BCNF), relationship cardinality and participation definitions.
- 03** [MySQL 8.0 Reference Manual](#)
Cited for: ENUM data type behaviour, CHECK constraint support (MySQL 8.0.16+), ON DELETE/ON UPDATE FK rules, DECIMAL precision semantics.
- 04** [Mermaid.js Entity Relationship Diagrams documentation](#)
Cited for: erDiagram syntax, crow's foot notation conventions, attribute type declarations in Mermaid v9+.

D — GLOSSARY

1NF (First Normal Form)	A relation is in 1NF if all attributes contain only atomic (indivisible) values, there are no repeating groups, and every row is uniquely identifiable by a primary key.
3NF (Third Normal Form)	A relation is in 3NF if it is in 2NF and no non-key attribute transitively depends on another non-key attribute — every non-key attribute depends directly on the primary key and nothing else.
BCNF (Boyce-Codd Normal Form)	A stricter version of 3NF: for every non-trivial functional dependency $X \rightarrow Y$ in the relation, X must be a superkey (a key or superset of a key). All tables in this schema satisfy BCNF.
Bridge / Associative Entity	A table that resolves a many-to-many relationship by holding foreign keys to both parent tables, a surrogate PK, and optionally attributes of the relationship (e.g. role, price snapshot, assignment date).
Controlled Denormalisation	A deliberate deviation from strict normal form where a derived or redundant column is stored for performance reasons and maintained transactionally to prevent inconsistency. Invoice.amount_paid is the sole example in this schema.
Crow's Foot Notation	An ERD notation where a single vertical bar means exactly one, a circle means zero, and a three-pronged crow's foot means many. Combinations express cardinality: --o{ means one-to-zero-or-many.

Surrogate Key	An artificially generated primary key (typically INT AUTO_INCREMENT) that has no business meaning. Used throughout this schema to decouple FK references from changes in natural business identifiers.
Total Participation	Every instance of an entity must participate in the relationship. Shown as a double line in Chen notation; modelled as a NOT NULL FK in the relational schema.
Partial Participation	Some instances of an entity may not participate in the relationship. Shown as a single line in Chen notation; modelled as a nullable FK or a zero-or-many crow's foot in the relational schema.